

The Ultimate Master Guide: Streaming Unity to Apple Vision Pro via NVIDIA CloudXR on Google Cloud (Secure Edition)

Quick Summary Checklist

The final configuration we built together:

- Server Engine: NVIDIA CloudXR SDK 6.0.4/6.0.5 (Stream Manager).
- Host Environment: Google Cloud VM (GCP) with Tesla T4 (Note: Upgrade to L4/L40 recommended for 2026 standards).
- Game Hook: Unity OpenXR via `XR_RUNTIME_JSON` environment variable.
- Security Mode: LocalSecure (Token-based authentication + Certificate bypass).
- Client App: Custom Xcode visionOS App using `CloudXRKit`.

Phase 1: Google Cloud Network Prep (Do this first)

Since you are streaming over the open internet from a cloud VM, the network gates must be opened.

1. Go to your **Google Cloud Console** web dashboard.
2. Navigate to **VPC Network > Firewall**.
3. Create a rule named `allow-cloudxr`:
 - **Targets:** All instances in the network.
 - **Source IP ranges:** `0.0.0.0/0`
 - **Protocols and ports:** * TCP: `48010`, `48322`
 - UDP: `48011`, `48012`
4. On your actual Windows VM, ensure **Windows Defender Firewall** is either turned off for testing or explicitly allowing those same ports.

Phase 2: The Batch File (The "Glue")

You must use a batch file to launch your game so that it uses the CloudXR engine instead of looking for a physically plugged-in VR headset.

1. On your VM, navigate to your game folder: `C:\New folder\`
2. Right-click and edit your `Start_CloudXR.bat` file to contain exactly this text:

DOS

@echo off

echo Setting OpenXR Runtime to NVIDIA CloudXR...

set

XR_RUNTIME_JSON=C:\Users\jan_armin_steinhauer\Downloads\Stream-Manager-6.0.4-win64 - Copy\Server\releases\6.0.5\openxr_cloudxr.json

echo Starting Unity Game...

start "" "CloudXRStreamingTest.exe"

echo Game launched! Waiting for AVP connection...

pause

Save and close this file.

Phase 3: Start the Server & Generate Credentials

This is the routine you will run every time you start up the server.

1. Open File Explorer and navigate to:
`C:\Users\jan_armin_steinhauer\Downloads\Stream-Manager-6.0.4-win64 - Copy\Server`
2. Double-click `NvStreamManager.exe`. Leave this black command prompt open in the background.

Open a **PowerShell** window and navigate to your client folder:

PowerShell

cd "C:\Users\jan_armin_steinhauer\Downloads\Stream-Manager-6.0.4-win64 - Copy\SampleClient"

3.

Run the tool:

PowerShell

.\SampleNvStreamManagerClient.exe

4.

Register your headset to get your token (copy the token it gives you!):

PowerShell

SetClientId AppleVisionPro

5.

Start the streaming engine:

PowerShell

StartCxrService 6.0.5

6. *Wait until it says "CloudXR service start request successful."*

Phase 4: Launch the Game

1. Go back to `C:\New folder\`.
 2. Double-click `Start_CloudXR.bat`.
 3. Your game will open.
 4. *Optional Check:* In your PowerShell window, type `GetCxrServiceStatus`. It should now say **OpenXR App: Connected**.
-

Phase 5: The Apple Vision Pro Client

To connect securely over the internet, we bypass the strict Apple certificate check and feed it your specific VM credentials.

1. Open **Xcode** on your Mac.
2. Open `ContentView.swift`.
3. Use the following code for your Connect button. **Crucial:** Make sure you update the `ip` with your VM's current Public IP address, and the `clientToken` with the token you generated in Phase 3.

Swift

```
Button("Connect") {
    Task { @MainActor in
        var config = CloudXRKit.Config()

        // Replace with your current Google Cloud Public IP and Token!
        config.connectionType = .localSecure(
            ip: "34.158.227.20",
```

```

clientToken: "i0sIx+xb0vYS/D0SkKCD2VSeK5XFnCCpN6ZYUNmaqQQ",
certificateValidationHandler: { challenge in
    // This tells the AVP to trust your Google Cloud VM's connection
    if let trust = challenge.protectionSpace.serverTrust {
        return (.useCredential, URLCredential(trust: trust))
    }
    return (.performDefaultHandling, nil)
}
)

config.resolutionPreset = .standardPreset
cxrSession.configure(config: config)

// Connect!
try await cxrSession.connect()

await openImmersiveSpace(id: streamingSpaceTitle)
}
}.padding()

```